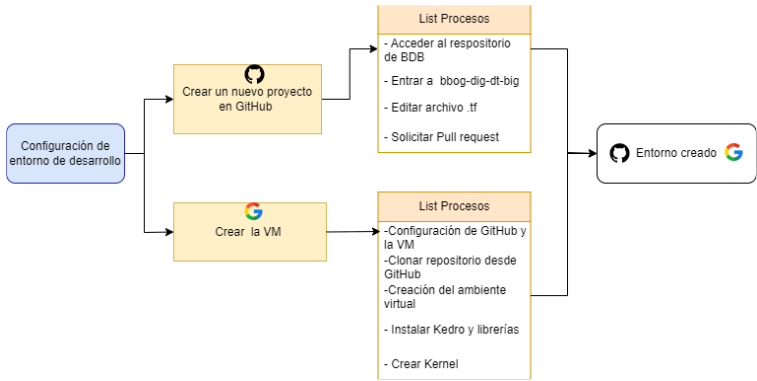


Configuración de entorno de desarrollo

A lo largo de esta documentación, aprenderán los pasos esenciales para configurar un entorno de desarrollo, aprovechando las potentes capacidades de GCP y las prácticas recomendadas de GitHub. Veremos cómo estos elementos se combinan de manera sinérgica con Kedro, facilitando la creación y gestión de proyectos de ciencia de datos de manera eficiente y organizada.

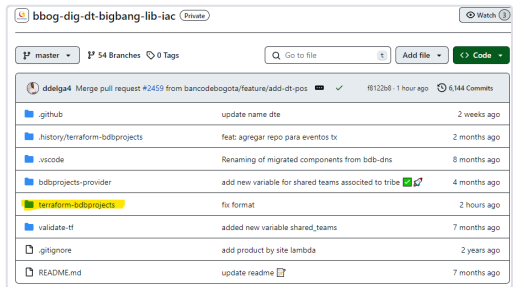
Exploraremos de manera detallada el proceso de creación de máquinas virtuales en Google Cloud Plataforma (GCP) utilizando Vertex AI. Además, se proporcionará una guía completa sobre cómo establecer una conexión segura con GitHub por medio de una llave SSH, integrando eficazmente la arquitectura de Kedro para gestionar código de manera modular.



Creación de nuevo proyecto en GitHub

A continuación se mostraran los pasos necesarios para crear un nuevo proyecto en GitHub:

1. Acceder al repositorio del Banco de Bogotá con su cuenta institucional [bancodebogota](#)
2. Ir al repositorio en GitHub de Big Bang, que es la herramienta de generación automática de proyectos del área de Digital y buscar el siguiente repositorio: bbog-dig-dt-big. Pueden acceder por medio del siguiente enlace [bancodebogota/bbog-dig-dt-bigbang-lib-iac](#)
3. Se selecciona la carpeta [terraform-bdbprojects](#), dentro de dicha carpeta se debe buscar el el archivo correspondiente a su equipo que estará denotado por la palabra **team-*susfijo*.tf** relacionado al equipo



Con fines prácticos y para el desarrollo del tutorial se ha seleccionado el equipo de Analítica Avanzada el cual corresponde a **team-aad.tf**

4. Abra el archivo **team-aad.tf** en la carpeta [terraform-bdbprojects](#) y seleccione la opción de editar

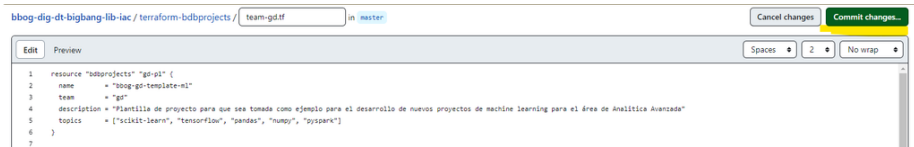


5. Escoja el nombre de repositorio/proyecto y sus parámetros conforme al lineamiento del área, en el caso de analítica el nombre del proyecto tiene la siguiente convención **"bbog-gd-nombre_del_proyecto-ml"** y agregue una nueva entrada de este estilo:

```
1 resource "bdbprojects" "gd-p1" {
2   name      = "bbog-gd-nombre_del_proyecto-ml"
3   team      = "gd"
4   description = "Plantilla de proyecto para que sea tomada como ejemplo para el desarrollo de nuevos proyectos de machine learning para el área de Analítica Avanzada"
5   topics    = ["scikit-learn", "tensorflow", "pandas", "numpy", "pyspark"]
6 }
```

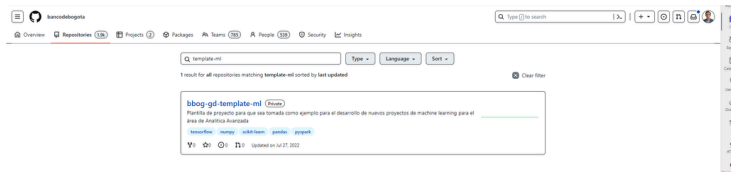
Donde el nombre **'bdbprojects'**, que representa el proveedor y no debe ser modificado. El identificador único del recurso, **'gd-p1'**, generalmente **refleja el nombre del archivo con un consecutivo adicional (p#)**. Dentro del campo **'name'** del recurso, se debe ingresar el nombre del proyecto siguiendo el formato (**bbog-gd-nombre_del_proyecto-ml**). Finalmente, el campo **'team'** corresponde al equipo o squad, en este caso, **'gd'**.

4. Al finalizar se debe guardar presionando el botón de **“Commit changes...”** esto generará una ventana en la cual permite agregar un mensaje descriptivo relacionado al cambio que se acaba de realizar. Este mensaje será cargado por medio del botón **Propose changes**.



5. Después de aceptar estos cambios se refrescará la pagina realizando un pull request el cual se debe solicitar al equipo la revisión y aprobación del mismo. Se necesitan mínimo dos aprobadores para finalizar el proceso.

6. Después de unos minutos aparecerá en el repositorio del banco el nuevo proyecto creado



En el siguiente enlace puede encontrar mas información acerca de la creación de proyectos en GitHub , su arquitectura y manuales de usuario: [Building Block - BigBang Generator](#)

Si quieren conocer mas de GitHub [Git - GitHub](#)

Creación y configuración de VM en Vertex AI

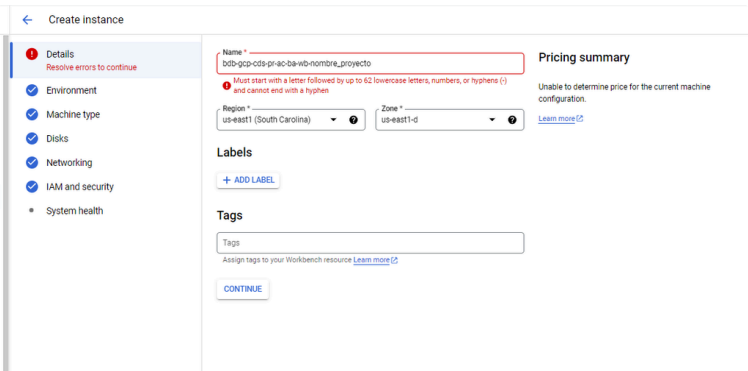
Existen diversas formas para la creación de maquinas virtuales en GCP. Se pueden crear máquinas virtuales en Vertex AI a través de comandos desde [Cloud Shell](#) o directamente desde Workbench.

Creación de una VM por medio de Workbench

Para crear una VM atraves de Workbench se deben seguir los siguientes pasos :

Las imágenes son tu mejor guía en esta etapa, así que adelante y sigue el paso a paso visual para una configuración perfecta.

1. Ingresar a GCP y entrar al tablero de Vertex AI seleccionando el notebook de Workbench: [Vertex AI/Workbench](#)
2. Para crear una nueva VM solo es necesario seleccionar la opción **+ CREATE NEW** que se encuentra en la parte superior de la página.
3. Se va a desplegar un menú en la parte derecha de la página y se va a seleccionar la opción de **“ADVANCED OPTIONS”**
4. Ahora, serás dirigido a una nueva página donde necesitarás ingresar algunos datos clave, como detalles, entorno, tipo de máquina, entre otros. Para facilitar este proceso, te sugiero seguir las configuraciones mostradas en las imágenes correspondientes a cada sección.
 - a. **Details:** En el campo *Name* se de poner el nombre del proyecto según los lineamientos del equipo. En el caso del squad de Analítica el nombre del proyecto debe ser *“bdb-gcp-cds-pr-ac-ba-wb- nombre_proyecto”*. Los otros campos se deben dejar como aparecen en la imagen.



b. **Environment:** el sistema operativo se selecciona por defecto sin embargo en el campo de Environment o ambiente dependiendo el idioma que te aparezca puede ser diferente, en el caso de analítica y para desarrollos del area seleccionamos *“Python 3 (with Intel MKL)”*

- Details
- Environment
- Machine type
- Disks
- Networking
- IAM and security
- System health

Environment

All environments use JupyterLab 3 by default and have the latest NVIDIA GPU and Intel libraries and drivers installed. You can specify a previous version instead. [Learn more](#)

Operating system *
Debian 11

Environment *
Python 3 (with Intel® MKL)

Selected CUDA libraries provided if GPUs are selected. Includes key packages for handling data, such as scikit-learn, pandas, and NCTR.

Version
☒ Use the latest version
☐ Use a previous version
To learn more about specific versions, see the [Vertex AI Workbench release notes](#).

Post-startup script [BROWSE](#)

Cloud Storage path to script that automatically runs after the instance boots up

Metadata

Some metadata keys including data-disk-uri, enable-sshlogin, framework, notebook-api, notebook-api-version, nvidia-driver-gpu-path, proxy-uri, restrictions, shutdown-script, title, version are reserved for system use only. If you use these variable names below, they will be overwritten by system values.

[+ ADD METADATA](#)

[CONTINUE](#)

Pricing summary

\$57.61 monthly estimate
That's about \$0.079 hourly

Pay for what you use: No upfront costs and per second billing
Networking cost also applies. [Learn more](#)

[DETAILS](#)

c. **Machine type:** La elección del tipo de máquina es crucial y varía según las necesidades de cada proyecto. La determinación del tamaño del proyecto es fundamental, ya que determinará si se requiere un mayor número de CPU o más memoria RAM, lo que, a su vez, influye en los costos asociados. Por defecto, las primeras máquinas se crean con el tipo de serie **N1-standard-4**, pero es posible realizar modificaciones después de su creación según los requisitos específicos de cada proyecto.

- Details
- Environment
- Machine type
- Disks
- Networking
- IAM and security
- System health

Machine type

☒ General purpose ☐ Compute optimized ☐ Memory optimized ☐ GPUs

Machine types for common workloads, optimized for cost and flexibility

Series	Description	vCPUs	Memory
☐ C3	Consistently high performance	4 - 176	8 - 1,408 GE
☐ C3D	Consistently high performance	4 - 360	8 - 1,440 GE
☐ E2	Low cost, day-to-day computing	2 - 32	4 - 128 GB
☐ N2	Balanced price & performance	2 - 128	4 - 864 GB
☐ N2D	Balanced price & performance	2 - 224	4 - 896 GB
☐ T2A	Scale-out workloads	1 - 48	4 - 192 GB
☐ T2D	Scale-out workloads	1 - 60	4 - 240 GB
☒ N1	Balanced price & performance	1 - 96	3.6 - 624 GB

Machine type
n1-standard-4 (4 vCPU, 15 GB memory)

vCPU 4 Memory 15 GB

[CPU PLATFORM AND GPU](#)

Shielded VM

Turn on all settings for the most secure configuration. [Learn more](#)

☐ Secure Boot
Verifies the digital signature of all boot components to protect against boot-level and kernel-level malware. Not supported for instances with GPUs.

☒ Virtual Trusted Platform Module (vTPM)
Validates guest VM pre-boot and boot integrity, and offers key generation and protection.

☒ Integrity monitoring
Monitors and verifies runtime boot integrity using Stackdriver. Requires vTPM.

Idle shutdown

☐ Enable Idle Shutdown

[CONTINUE](#)

Pricing summary

\$110.32 monthly estimate
That's about \$0.151 hourly

Pay for what you use: No upfront costs and per second billing
Networking cost also applies. [Learn more](#)

[DETAILS](#)

d. **Disks :** Este apartado esta configurado por defecto , así que solo se le debe dar continuar.

- Details
- Environment
- Machine type
- Disks
- Networking
- IAM and security
- System health

Disks

Boot disk type
Standard Persistent Disk

Boot disk size in GB *
100

Data disk type
Standard Persistent Disk

Data disk size in GB *
100

☐ Delete to trash
Deleting in JupyterLab will use the operating system's trash behavior

Backup

Backup syncs a Cloud Storage location to your instance's data disk. [Charges apply](#)

Here's how it works:

- On instance creation, any files and folders in the selected location will be copied to the data disk
- After creation, any changes (including deletion) made to the data disk contents will be synced to the Cloud Storage location

It's recommended that you enable [object versioning](#) for the selected location so you can restore accidentally deleted or overwritten files. To prevent sync conflicts, avoid assigning the same location to multiple instances.

Choose Cloud Storage location [BROWSE](#)

Encryption

☒ Google-managed encryption key
No configuration required

☐ Customer-managed encryption key (CMK)
Manage via [Google Cloud Key Management Service](#)

[CONTINUE](#)

Pricing summary

\$57.61 monthly estimate
That's about \$0.079 hourly

Pay for what you use: No upfront costs and per second billing
Networking cost also applies. [Learn more](#)

[DETAILS](#)

e. **Networking:** Se debe seleccionar la opción de Shared Network la cual cambiara las siguientes opciones por defecto. Además, es muy importante que la opción de "Assing ecternal IP Adress" este **deshabilitada** , ya que esta opción es la que nos permite la conexión de la máquina a internet.

f. **IAM and security:** Esta configuración es por defecto solo se debe seleccionar el botón de continuar

g. **System health:** Por ultimo se encuentra la ventana de monitoreo la cual viene por defecto, lo que se debe hacer en este apartado es seleccionar el botón de crear.

Con esto hemos terminado la lista de pasos para la creación de las maquinas virtuales por medio de WorkBench

Configuración de git en la VM [↗](#)

Procederemos a configurar la VM con su respectivo repositorio desde la ventana de terminal de la máquina. Para ingresar a esta opción solo debemos seleccionar la opción **"OPEN JUPYTERLAB"** de la VM creada y seleccionar la opción *Terminal*.

Cuando ingresemos a la ventana del terminal se deberá seguir los siguientes pasos:

1. Configurar nuestro nombre y correo corporativo (el correo debe coincidir con el de la cuenta de GitHub corporativa)

```
1 git config --global user.email "tu@email.com"
2 git config --global user.name "Tu Nombre"
```

Si por algún motivo te equivocaste en el nombre o email que configuraste al principio, lo puedes modificar de la siguiente manera:

```
git config --global --replace-all user.name "Aquí va tu nombre modificado"
```

O si lo deseas eliminar y añadir uno nuevo

```
git config --global --unset-all user.name
git config --global --add user.name "Aquí va tu nombre"
```

2. Configurar la llave ssh, cuando aparezca `Enter file in which to save the key`, `Enter passphrase` y `Enter same passphrase again` pulsa `Enter` sin escribir nada.

```
1 ssh-keygen -t rsa -b 4096 -C "tu@email.com"
```

Aparecerá una ruta dónde ha sido guardada la identificación y la llave pública. Esta ruta será importante en el paso 4

```
Your identification has been saved in /home/jupyter/.ssh/id_rsa
Your public key has been saved in /home/jupyter/.ssh/id_rsa.pub
```

3. Encender el "servidor" de llaves SSH de tu computadora:

```
1 eval $(ssh-agent -s)
```

4. Añadir tu identificación SSH a este "servidor", copia la ruta de identificación del paso 2.

```
1 ssh-add ruta-donde-guardaste-tu-llave-privada
2
3 Ejemplo:
4 ssh-add /home/jupyter/.ssh/id_rsa
```

En este punto la máquina virtual reconoce que esa llave pertenece a tu correo y GitHub podrá reconocer el usuario.

5. En GitHub ir a settings (configuración) > SSH and GPG Keys (Llaves SSH y GPG)

 <https://github.com/settings/keys> Can't find link

SH keys associated with your account. Remove any keys that you do not recognize.

Keys

Dar click en New SSH key (crear nueva llave SSH)

Title

Key type

Authentication Key ▾

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

Add SSH key

Añade un nombre (Title) para la llave, se recomienda que lleve el nombre de la máquina virtual.

6. Regresa a la máquina virtual y ejecuta la siguiente instrucción. Aquí estamos abriendo la llave pública del túnel SSH.

```
1 cat /home/jupyter/.ssh/id_rsa.pub
```

Pega la llave pública generada y cópiala en el campo key en GitHub.

7. Una vez creada la llave, aparecerá de la siguiente manera.

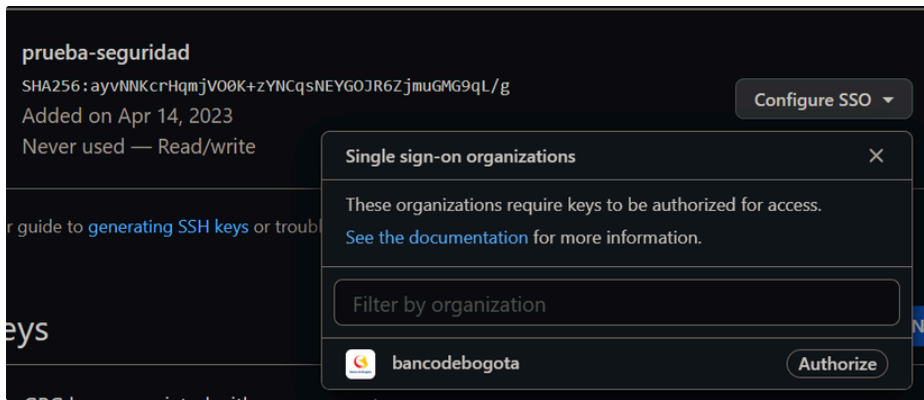


prueba-seguridad
SHA256:ayvNNKcrHqmjV00K+zYNCqsNEYG0JR6ZjmuGM69qL/g
Added on Apr 14, 2023
Never used — Read/write

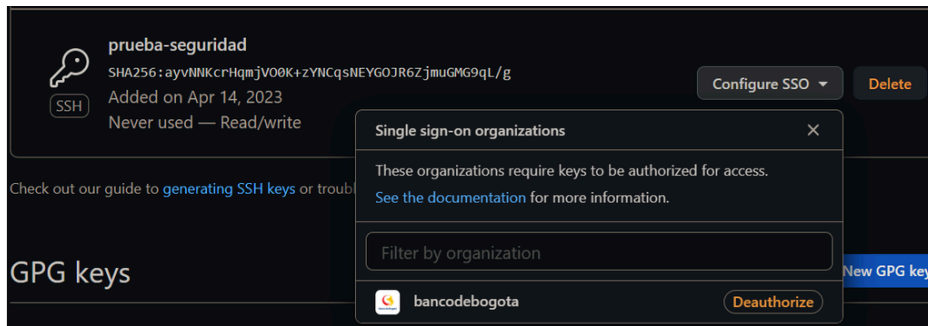
Configure SSO ▾

Delete

8. Ahora debes autorizar el acceso de esta llave a la organización bancodebogota. Da click en Configure SSO y luego en Authorize.



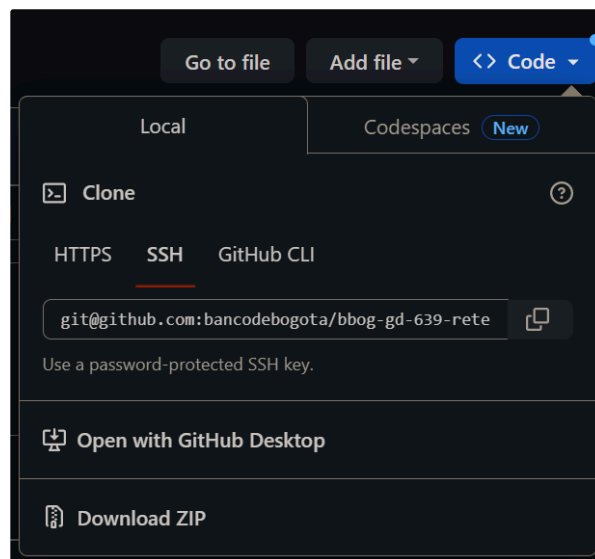
Luego de autenticarnos para la autorización se debería ver así



Clonar el repositorio del proyecto desde GitHub [↗](#)

1. Una vez configurada la llave SSH, se puede clonar el repositorio remoto en la máquina virtual:

Vaya al repositorio de su proyecto en GitHub y de clic en el botón Code. Seleccione SSH. Copie la URL SSH.



2. Vaya a su máquina virtual y utilice el siguiente comando, reemplace por el URL obtenido en el paso anterior.

```
1 git clone url-ssh-del-repositorio-en-github
```

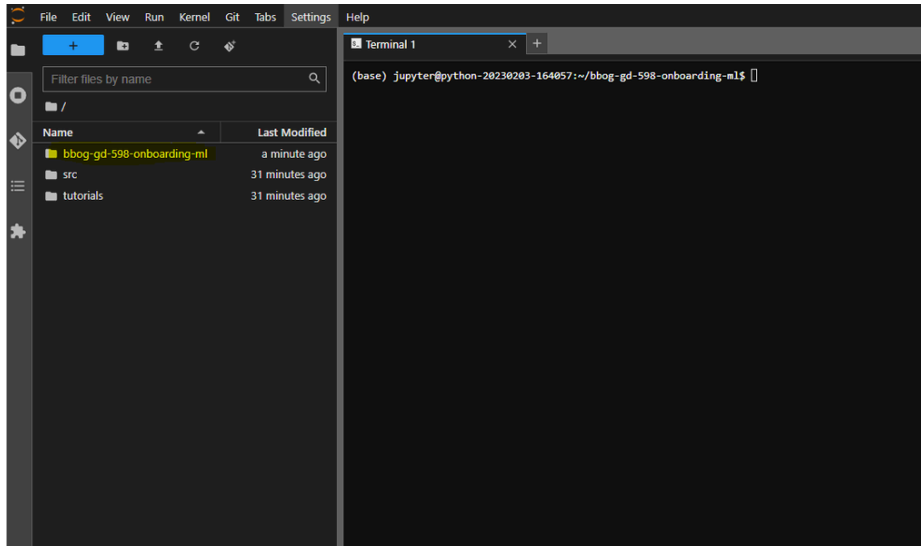
Es posible que aparezca el siguiente aviso

```
1 The authenticity of host 'github.com (140.82.114.4)' can't be established.
2 Are you sure you want to continue connecting (yes/no)?
```

Escriba yes y pulse Enter.

```
1 yes
```

Si la clonación ha sido exitosa podrá observar una carpeta en el Explorador de Archivos.



3. Podemos ingresar a este repositorio en la terminal con el comando `cd nombre-carpeta`, correspondiente al nombre del repositorio creado

4. Para validar, ejecute `git status` dentro de la carpeta del proyecto, debería obtener lo siguiente:

```
Terminal 1
(base) jupyter@python-20230203-164057:~/bbog-gd-598-onboarding-ml$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

Valide también todas las ramas locales y remotas usando `git branch -a`

```
master
remotes/origin/HEAD -> origin/master
remotes/origin/master
remotes/origin/qa
remotes/origin/staging
```

Traer el ultimo repositorio del proyecto desde GitHub [↗](#)

Para ver el ultimo con su respectivo mensaje commit

```
1 git log -1
```

1. Identificar la rama en la que estas

```
1 git branch
```

2. Establecer la rama de seguimiento: Esto establecerá la rama remota que tu rama local debe seguir.

```
1 git branch --set-upstream-to=origin/<nombre_de_la_rama>
2
3 ejemplo
4 git branch --set-upstream-to=origin/feature/trainpipeline
```

3. Hacer un `git pull` a la rama deseada del repositorio remoto:

```
1 git pull
```

Creación de virtual environment [↗](#)

1. Se debe crear un ambiente virtual con Conda antes de empezar cada proyecto, para esto ejecutaremos el siguiente comando en la terminal:

```
1 conda create --name nombre-ambiente python=3.9 -y
```

Se recomienda que el nombre del ambiente contenga la versión de Python utilizada y el nombre del proyecto, por ejemplo: `py39_nombre_proyecto`

Para activar o desactivar el ambiente virtual se usan los siguientes comandos:

```
1 # To activate this environment, use
```

```
2 conda activate nombre-ambiente
3
4 # To deactivate an active environment, use
5 conda deactivate
```

i Cada vez que se encienda la VM se debe activar el ambiente virtual

En la línea de comandos se muestra el ambiente activo:

2. Actualizar conda con:

```
1 conda install -n nombre-ambiente -c conda-forge conda=22.9.0
```

i Estos dos comandos actualizan las versiones de Python y Conda instaladas por default en las VM.

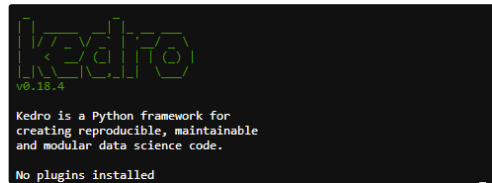
Se debe trabajar con Python>=3.9

Instalar e iniciar Kedro [↗](#)

1. Una vez activado el ambiente virtual, se instala Kedro con pip:

```
1 pip install kedro==0.18.4
```

Para validar que la instalación fue exitosa se puede ejecutar el comando `kedro info` en la línea de comandos, la cual debe mostrar la siguiente imagen:



2. Antes de crear un nuevo proyecto de Kedro primero debemos verificar que estamos dentro del repositorio que clonamos de GitHub, lo cual se puede validar en la terminal:

3. También se debe crear primero una nueva rama de feature a partir de la rama de qa:

```
1 #Para movernos a la rama de qa
2 git checkout qa
3
4 #Para crear nueva rama feature a partir de qa (ej: crear rama "feature/trainpipeline")
5 git checkout -b feature/trainpipeline
```

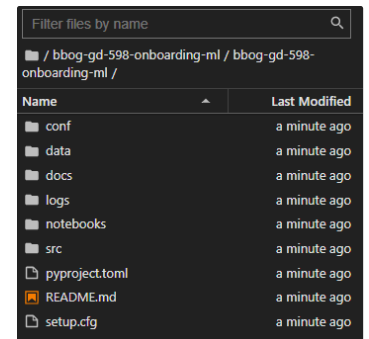
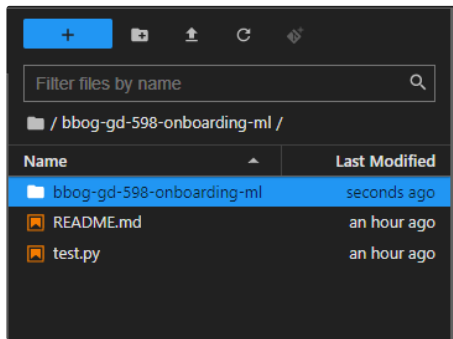
Con `git branch` podemos validar las ramas locales.

4. Una vez estamos en la nueva rama de feature, podemos crear el proyecto de Kedro con `kedro new`, el cual va a consultarnos el nombre del proyecto:

```
1 kedro new
2
3 Project Name
4 =====
5 Please enter a human readable name for your new project.
6 Spaces, hyphens, and underscores are allowed.
7 [New Kedro Project]:
```

i Importante que coloquemos el mismo nombre del repositorio de GitHub

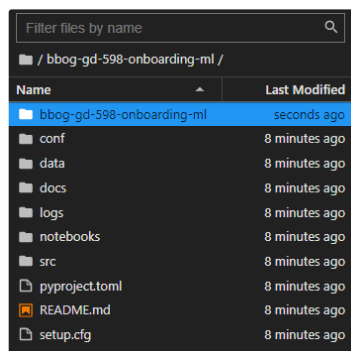
En este punto se debe haber creado una carpeta nueva, la cual contiene todos los archivos iniciales del proyecto:



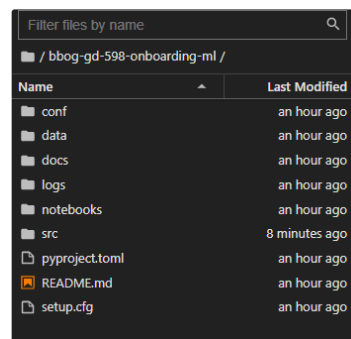
5. Ahora vamos a eliminar los archivos que están fuera de la carpeta creada por Kedro. Esto se puede hacer manualmente dando clic derecho en cada uno (en este caso `README.md` y `test.py`).
6. Para finalizar, vamos a mover todos los archivos que están dentro de la carpeta creada por Kedro hacia su carpeta padre, ya que son redundantes. Esto lo podemos hacer rápidamente con el siguiente comando en la terminal donde debemos reemplazar solo el nombre del proyecto sin espacios:

```
1 mv /home/jupyter/nombre-proyecto/nombre-proyecto/{.,}* /home/jupyter/nombre-proyecto/
```

En este punto el repositorio se debería ver así:



La carpeta resaltada en azul debe estar vacía y podemos eliminarla manualmente dando clic derecho en la misma. Finalmente tenemos nuestro repositorio listo y sin inconvenientes para empezar a trabajar y a hacer versionamiento de código con git y GitHub.



Instalar requirements.txt [↗](#)

Kedro genera por defecto un archivo `src/requirements.txt` para especificar las dependencias del proyecto. Es importante revisar este archivo para borrar las que aparecen por defecto y añadir las siguientes librerías que serán de utilidad en el futuro:

```
1 black==22.0
2 flake8>=3.7.9, <5.0
3 ipython>=7.31.1, <8.0
4 isort==5.0
5 jupyter==1.0
6 jupyterlab_server>=2.11.1, <2.16.0
7 jupyterlab==3.0
8 kedro==0.18.4
9 kedro-telemetry==0.2.0
10 nbstripout==0.4
11 pytest-cov==3.0
12 pytest-mock>=1.7.1, <2.0
13 pytest==6.2
14
15
16 #Para trabajar con este tipo de datasets (ver documentacion)
17 kedro[pandas, pickle, plotly, matplotlib]==0.18.4
```

```
18
19 scikit-learn==1.2.1
20 imbalanced-learn==0.1
21 pandas-gbq==0.1
22 pandas-profiling==3.6
23 google-cloud-aiplatform==1.2
24 feature-engine==1.5
25 gcsfs ==2022.1.0
26 xgboost ==1.7.6
```

✔ No olvides dar control + s , para guardar los cambios sobre el archivo

Se pueden agregar otras librerías que se usen en el desarrollo del proyecto. Por ejemplo:

```
1 #Para visualizar pipelines de Kedro
2 kedro-viz==5.0
3
```

❗ Importante validar las versiones de las librerías al momento de crear el proyecto e ir llevando el registro

Finalmente se ejecuta en la terminal:

```
1 pip install -r src/requirements.txt
```

Crear kernel para jupyter [↗](#)

Una vez instaladas las dependencias y activado el ambiente se debe crear un nuevo kernel de ipython para que Jupyter lo reconozca en un nuevo kernel. Se debe ejecutar el siguiente comando en CLI:

```
1 python -m ipykernel install --user --name nombre-environment --display-name "Python nombre-environment"
```

📺 En caso que tenga alguna duda puede revisar el siguiente video [2.2 Tutorial - Configuración entorno de nuevo proyecto](#)